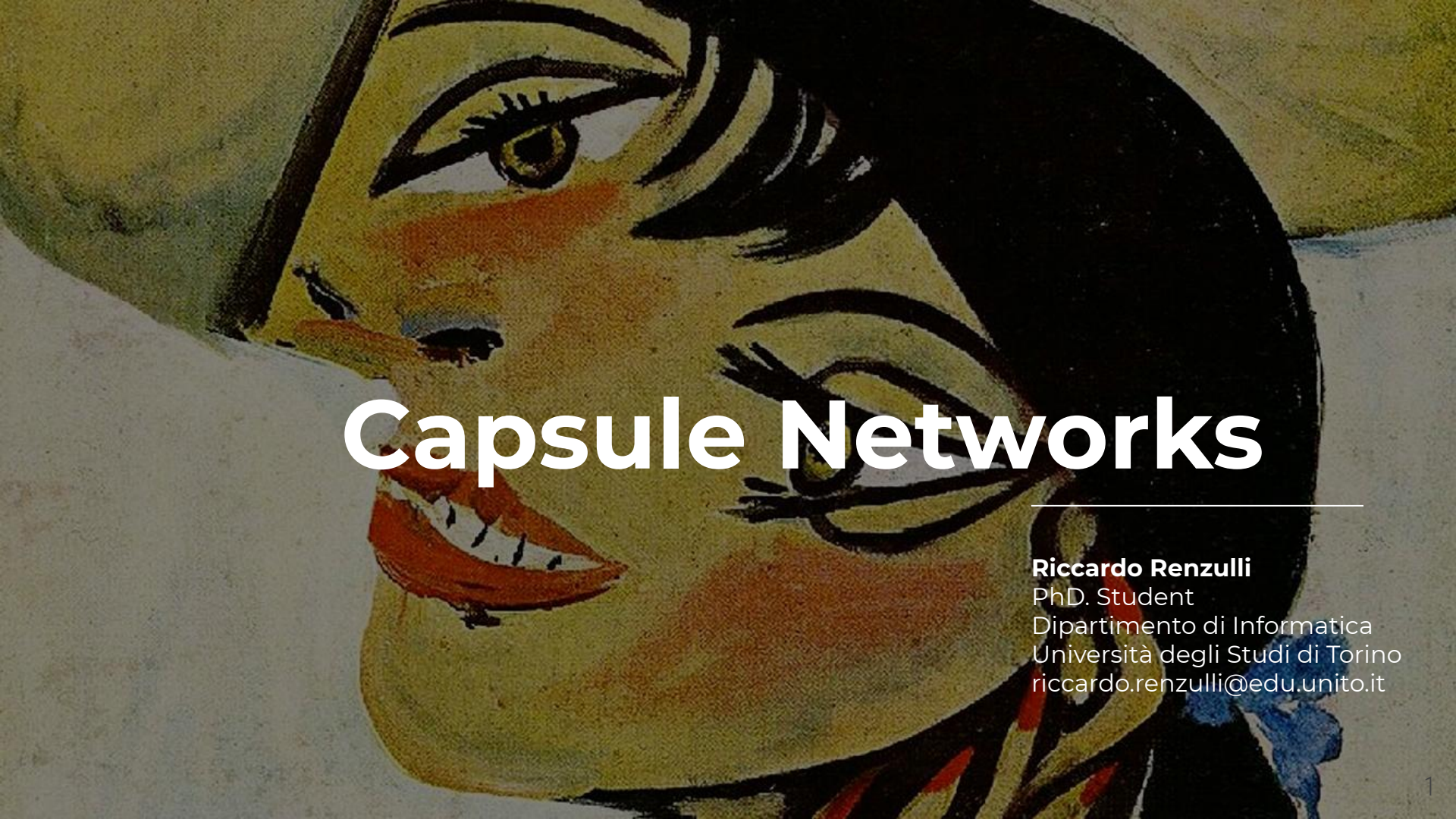


Capsule Networks

Riccardo Renzulli
PhD. Student
Dipartimento di Informatica
Università degli Studi di Torino
riccardo.renzulli@edu.unito.it

TABLE OF CONTENTS

01 From Perceptron to Deep Neural Networks

02 The limitations of Convolutional Networks

03 What are capsules?

04 Vector Capsule Networks

05 Matrix Capsule Networks

06 Pros and cons

01

From Perceptron to Deep Neural Networks

Perceptron

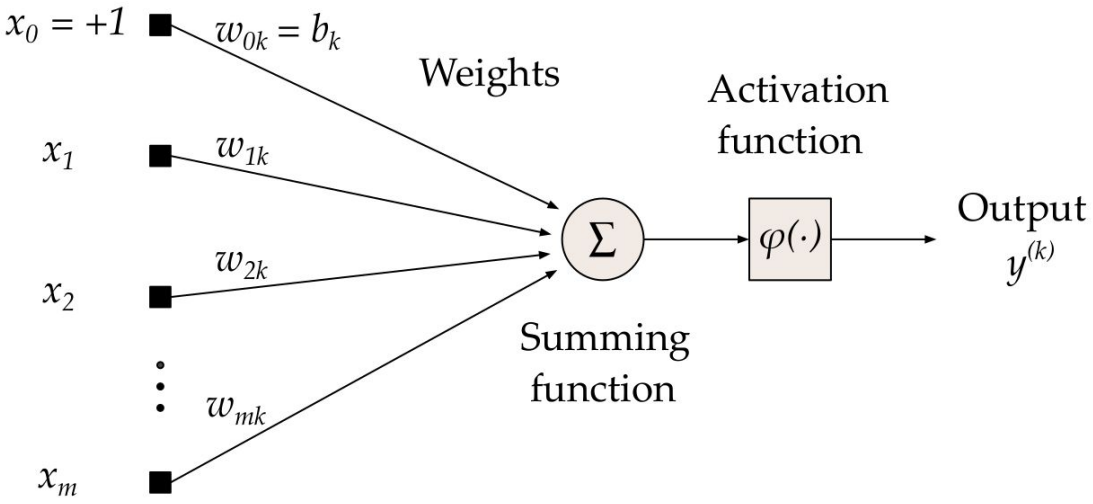


Fig 1. Basic nonlinear model of a neuron k.

Multilayer perceptron

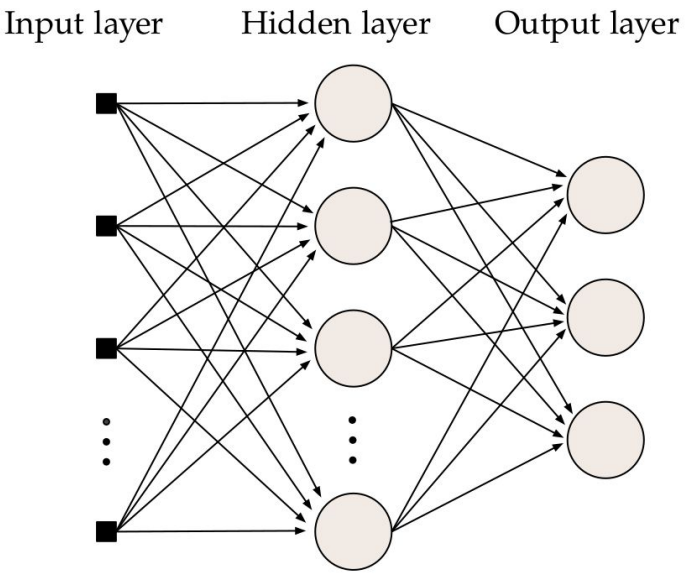


Fig 2. Feedforward network with one hidden layer.

Convolutional Neural Networks (CNNs)

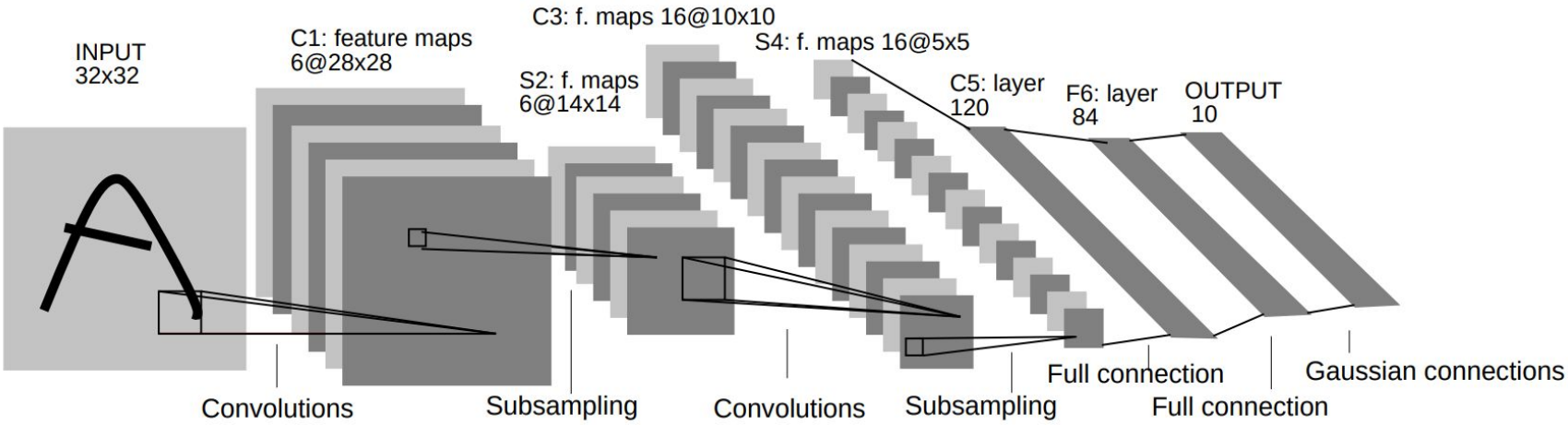


Fig 3. Architecture of LeNet-5 [1].

Convolution

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n)$$

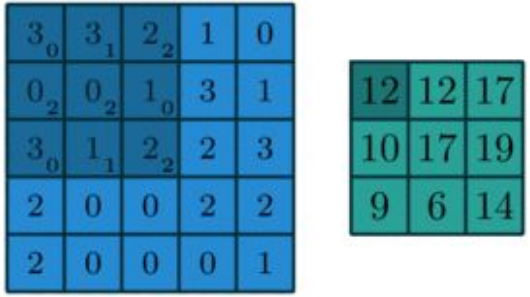


Fig 4. Convolution with 3x3 kernel, stride 1, no padding.
http://deeplearning.net/software/theano_versions/dev/tutorial/conv_arithmetic.html

Subsampling

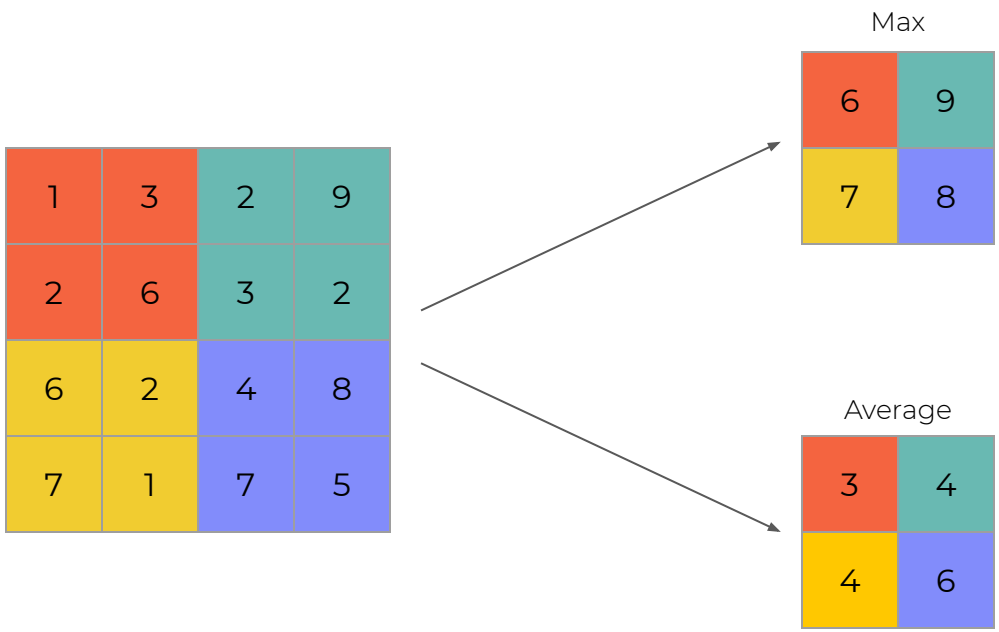


Fig 5. Max and average pooling with window size 2x2 and stride 2.

02

The limitations of CNN

CNN main characteristics

Convolutional networks have been remarkably successful in the last few years and they outperform other methods on object recognition tasks.

Their main characteristics are:

1. They use many learned **feature detectors** with different receptive fields.
2. Different types of feature detectors are **replicated** across space.
3. **Higher-level** feature detectors fields have **bigger** spatial dimensions.
4. Convolutional layers are interleaved with **pooling layers** that do **downsampling** on the input. They **combine** the outputs of nearby feature detectors neurons of the same type.

Max-pooling

“The fact that max-pooling works so well is extremely unfortunate because it makes it really hard to get rid of!”

"Geoffrey Hinton

Who is Geoffrey Hinton?

Geoffrey Everest Hinton (December 1947) is an English Canadian **cognitive psychologist** and **computer scientist**.

He is currently a professor in the computer science department at the **University of Toronto** and works at **Google**.

Main known for:

1. Boltzmann machines (1985).
2. Use of the backpropagation algorithm for training multi-layer neural networks (1986).
3. Deep Belief Networks (2006).
4. AlexNet (2012).
5. Capsule Networks (2017).

Citations	324430	217032
h-index	152	114
i10-index	372	275

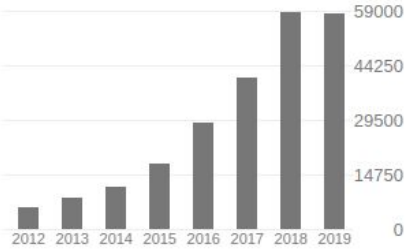


Fig 6. Hinton citations.
ttps://scholar.google.co.uk/citations?user=JicYPdAAAAAJ&hl=en

Who is Geoffrey Hinton?

Fig 7. Geoffrey Hinton and Sara Sabour.
https://www.nytimes.com/2017/11/28/technology/artificial-intelligence-research-toronto.html

Why max-pooling?

There are two main motivations behind pooling.

1. Throwing away the precise location of the most active neuron gives a small amount of **translation invariance** at each level.
2. Reducing the number of neurons to the next layer of feature extraction allows more types of features with **bigger spatial domains** at the next layer.
3. It **decreases spatial resolution**.

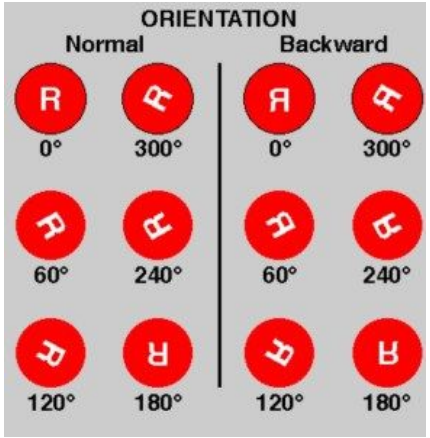
Why not max-pooling?

Hinton has four arguments against pooling.

1. It is a **bad fit** to the psychology of **shape perception**.
2. We want **equivariance**, **not invariance**.
3. It **fails** to use the underlying **linear structure**.
4. Pooling is a **poor way** to do **dynamic routing**.

Shape perception

Human vision system imposes rectangular **coordinate frames** on objects in order to represent them and it would perform mental rotation on the object to a point of reference which it's familiar to before making the comparison.



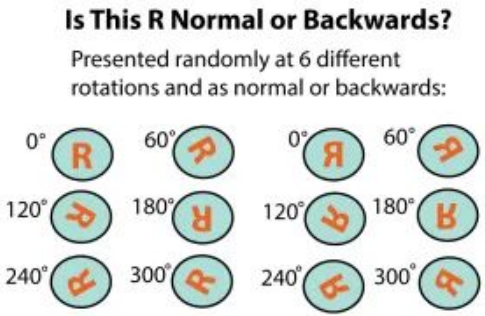


Fig 8. Different orientations.

https://principlesofdeeplearning.com/index.php/a-tutorial-on-capsule-networks/

Inverse computer graphics

"Our brain does a sort of **inverse computer graphics** transformation in order to do **shape perception**. Computer vision is inverse computer graphics, so the higher levels of a **vision system should look like the representations used in graphics**."

Geoffrey Hinton

Shape perception

Computer graphics has to say what the relation is between a part in the whole, so it put a frame on the part and tells you the **matrix** that will map point in the whole relative frame.

Computer graphics notation for **affine transformations** (translation, rotation, scaling, shearing, reflection, and orthogonal projection):

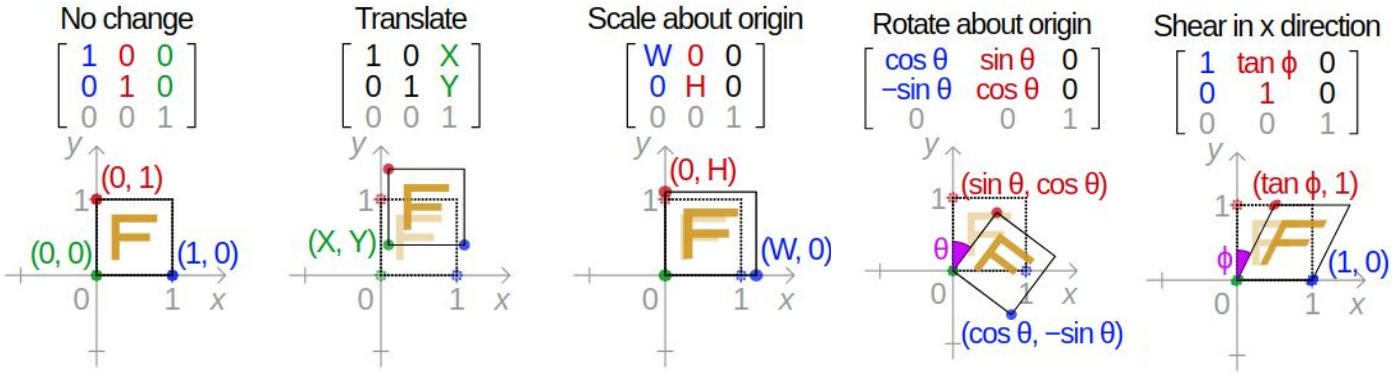


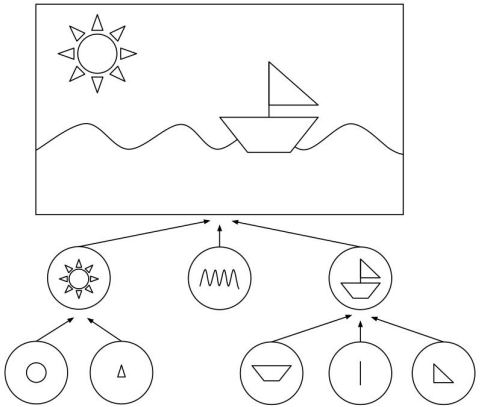
Fig 9. 3x3 transformation matrix in 2D. In 3D we have 4x4 matrices.

https://upload.wikimedia.org/wikipedia/commons/2/2c/2D_affine_transformation_matrix.svg

Shape perception

Graphics programs use hierarchical models in which spatial structure is modeled by **viewpoint invariant matrices** that represent the transformation from coordinate frame embedded in the whole to a coordinate frame embedded in each part.

Hierarchical modeling creates complex real-world objects by combining simple primitive shapes into more complex aggregate objects. There are psychological evidences that **shape perception in humans** works like this too.



$$\boldsymbol{T} = \begin{bmatrix} n_x & o_x & a_x & v_x \\ n_y & o_y & a_y & v_y \\ n_z & o_z & a_z & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Fig 10. A **scene graph**. Each connection represents one occurrence of the object in its parent object and it can be associated with a transformation T.

Shape perception

In CNNs there is no explicit notion of an **entity**. They do not use coordinate frames.

We would like to output the **pose** (position, size, orientation) of the entities in the image.

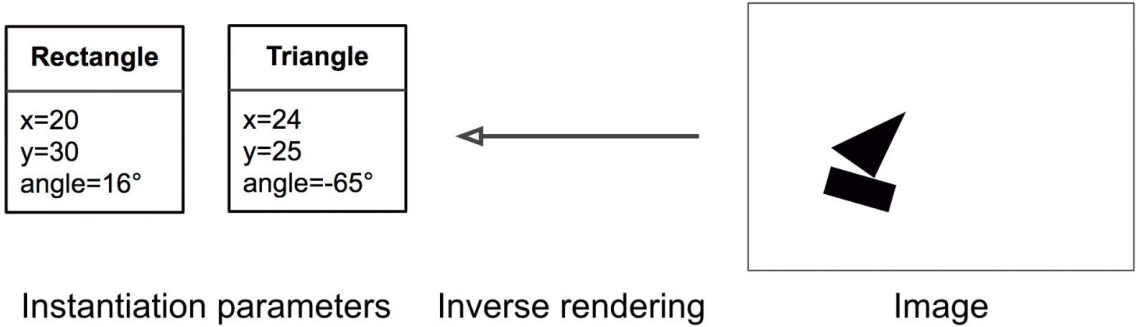


Fig 11. Inverse computer graphics. *https://www.youtube.com/watch?v=pPN8d0E3900*

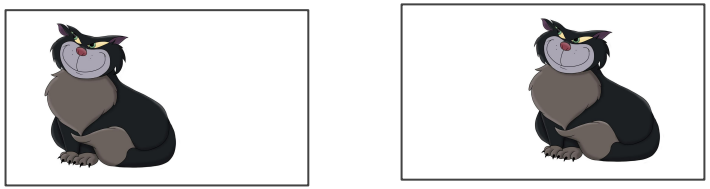
geoffhinton Google Brain 174 points · 1 year ago
This is an amazingly good video. I wish I could explain capsules that well.

nick_frosst Google Brain 63 points · 1 year ago
we all wish that Geoff :P

Equivariance vs Invariance

A function f is

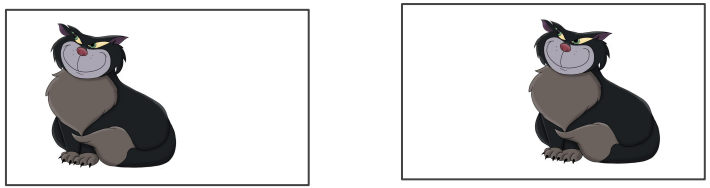
1. **invariant** with respect a transformation T if $f(T(x)) = f(x)$. If the input changes, the output does not change.



cat cat

Fig 12. Translational invariance.

2. **equivariant** with respect a transformation T if $f(T(x)) = T(f(x))$. If the input changes, the output changes in a coordinate way.



cat on the left cat on the right

Fig 13. Translational equivariance.

Equivariance vs Invariance

Hinton claims that it is better to aim for equivariance: we want representations where changes in viewpoint lead to corresponding changes in neural activities. For example, when we look at a face we do not merely recognize it but we also know exactly its position, orientation and scale.

With max-pooling we lose precise spatial relationships between objects.

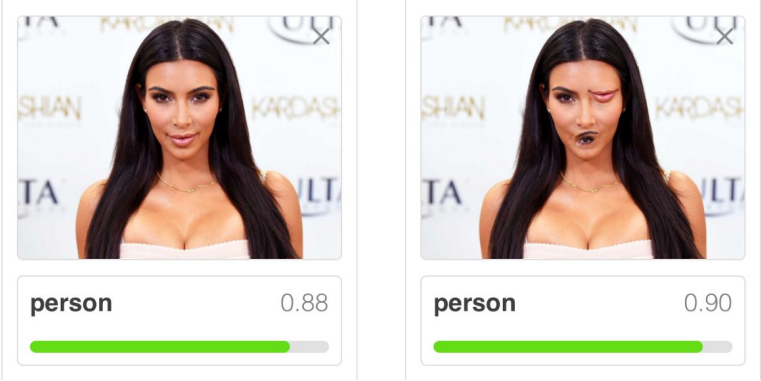


Fig 14. Person with right and wrong parts relationships.

https://hackernoon.com/capsule-networks-are-shaking-up-ai-heres-how-to-use-them-c233a0971952

Equivariance vs Invariance

Hinton distinguishes between two types of equivariance.

1. **place-coded**: A discrete change in a property of a visual entity leads to a discrete change in which neurons are used for encoding that visual entity.
2. **rate-coded**: A real-valued change in a property of a visual entity leads to a real-valued change in the output of some of the neurons used for coding that visual entity, but there is no change in which neurons are used.

Our vision systems may use both types.

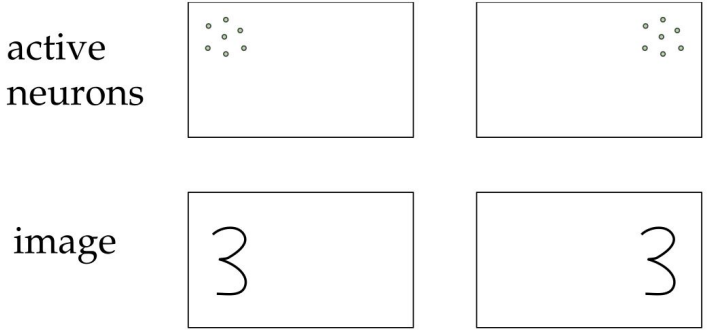


Fig 15. Example of place-coded equivariance when translating an image in a CNN without sub-sampling layers.

Equivariance vs Invariance

Convolution is **naturally equivariant to translation**.

It is **not naturally equivariant to other transformations**, such as scaling or rotation.

For example, if we rotate an image and we passed it through a CNN then the resulting feature maps are different from the ones obtained by passing the original input image and then rotating the feature maps.

$$f(T(x)) \neq T(f(x))$$

Our **networks** should be **robust to affine transformations**, otherwise they can be easily fooled.

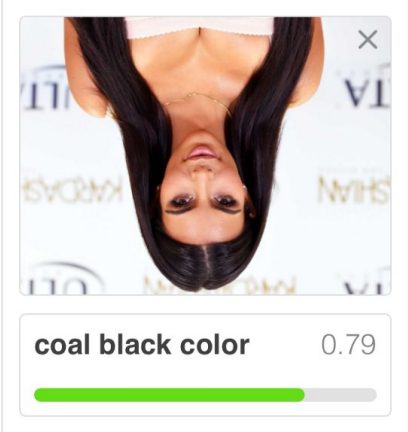


Fig 16. Wrong classification example of a rotated image.

https://hackernoon.com/capsule-networks-are-shaking-up-ai-heres-how-to-use-them-c233a0971952

Equivariance vs Invariance

CNNs try to make the **neural activities invariant to small changes in viewpoint** other than translation thanks to max pooling and multiple feature maps.

Max pooling over spatial positions is naturally **invariant to translation** (the max of a region depends only on the single largest element).

If we pool over the outputs of separately parametrized convolutions, the features can learn which transformations to become invariant to.

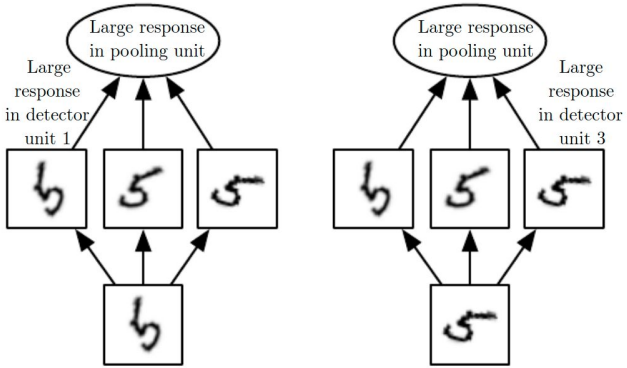


Fig 17. Example of a max pooling unit invariant to rotation.

http://www.deeplearningbook.org/contents/convnets.html

Equivariance vs Invariance

With CNNs we need more filters, data augmentation etc.

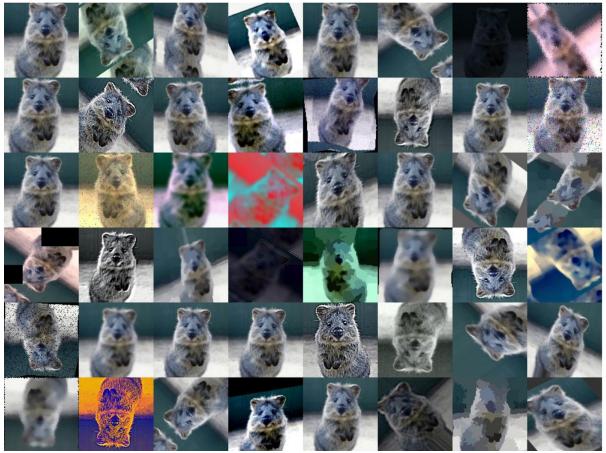


Fig 18. Example augmentations of one input image.

https://github.com/aleju/imgaug

More parameters, more features maps, bigger datasets!

Often real datasets do not contain images of the same object in different viewpoints. CNN tries to learn the rotated versions of the same image which increases the risk of overfitting.

Equivariance vs Invariance

Therefore, similarly to the perceptual system, in our deep learning models

1. **Viewpoint-invariant relationship between a part and a whole should be embedded in the weights**. The relation between coordinate frame of an object and its parts stays constant. For example, if you look at your nose while moving your head, you will see that the nose, relative to your eyes, is not moving at all.
2. **Knowledge of the varying relationship of each node to the viewer should be embedded in neural activities (equivariance)**.

This representation **improves generalization** requiring **less data and parameters**.

Routing

Hinton states that we need to route information in the image to the neurons that know how to deal with it. CNNs do a basic primitive routing by means of pooling: it is impossible to compute precise spatial relationships between objects.

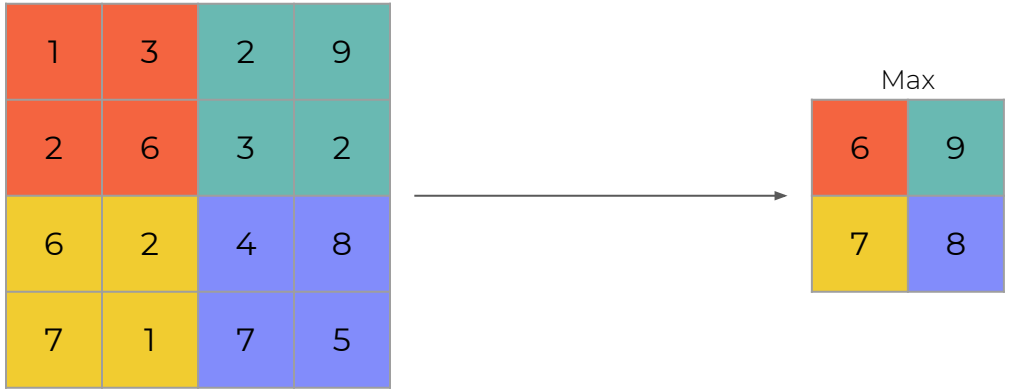


Fig 19. Max pooling with window size 2x2 and stride 2.

Finding the best routing is equivalent to parsing the image in order to obtain a parse tree.

CNNs drawbacks

1. In CNNs there is **no** explicit notion of an **entity**.
2. They **do not take into account** important **spatial hierarchies** between simple and complex objects.
3. They **require a lot of training data** in order to be **robust to affine transformations**.
4. CNNs do a **basic primitive routing** by means of pooling.

03

What are capsules?

Capsules

"**Instead** of aiming for viewpoint invariance in the activities **of** *neurons* that use **a single scalar output** to summarize the activities of a local pool of replicated feature detectors, **artificial neural networks should use local *capsules*** that perform some quite complicated internal computations on their inputs and then encapsulate the results of these computations into a small vector of highly informative outputs." [2]

Geoffrey Hinton

Capsules

A **capsule** is a **subset of neurons** in a layer.

Each capsule represents the **presence** and the **instantiation parameters** of an entity.

A capsule consists of two association parameters:

1. The **probability** p_i that an object of that type is present.
2. The generalized **pose** v_i (position, orientation, scale, deformation, velocity, color etc.).

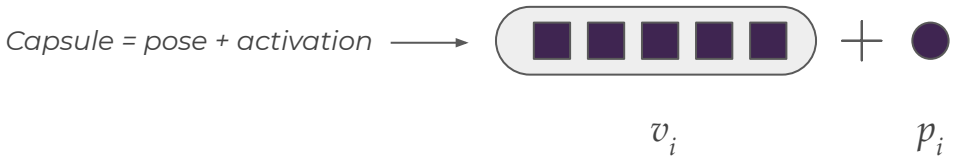


Fig 20. A capsule is a subset of neurons.

Capsules use **neural activities** that **vary as viewpoint varies** (they are equivariant).

The **probability of the visual entity being present is locally invariant**, it does not change as the entity moves in the image.

Capsules

Capsule are organized in layers. As in CNNs, higher-level capsules represent more complex entities and they cover larger regions of the image.

Capsules of level L make predictions, via transformation matrices, for the instantiation parameters of capsules of level L+1.

If these predictions are similar, it means that they are in the right spatial relationship to activate a higher-level capsule.

For example, according to the detected position of the eyes and the mouth, there must be a face in the image.

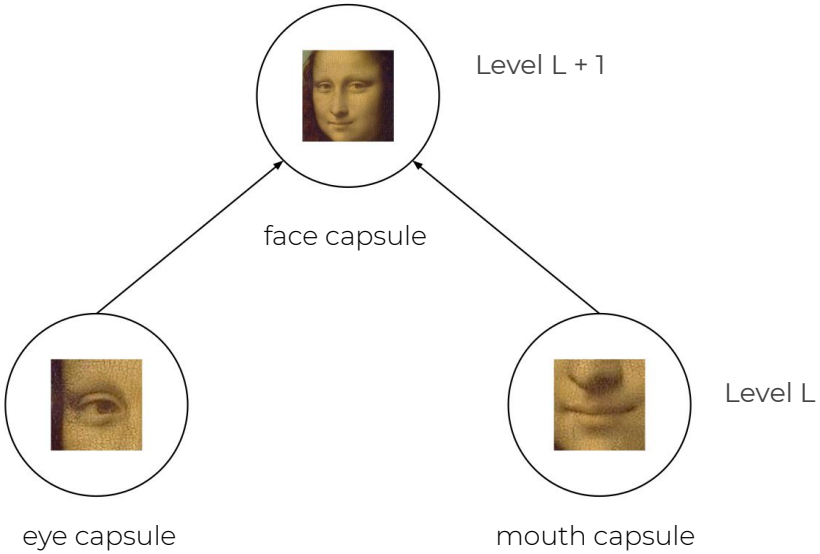


Fig 21. Hierarchical structure of capsules.

Capsules

From raw pixels we obtain the poses of the parts

Fig 22. How to compute the pose of a high-level capsule.

Capsules

From raw pixels we obtain the poses of the parts, we multiply this poses by transformation matrices W (our weight matrices learnt through backpropagation) to get the predictions for the pose of the whole.

W_{ik} = viewpoint-invariant weight matrix

$\hat{u}_{k|i} = W_{ik} v_i$ = prediction for capsule pose k from capsule i

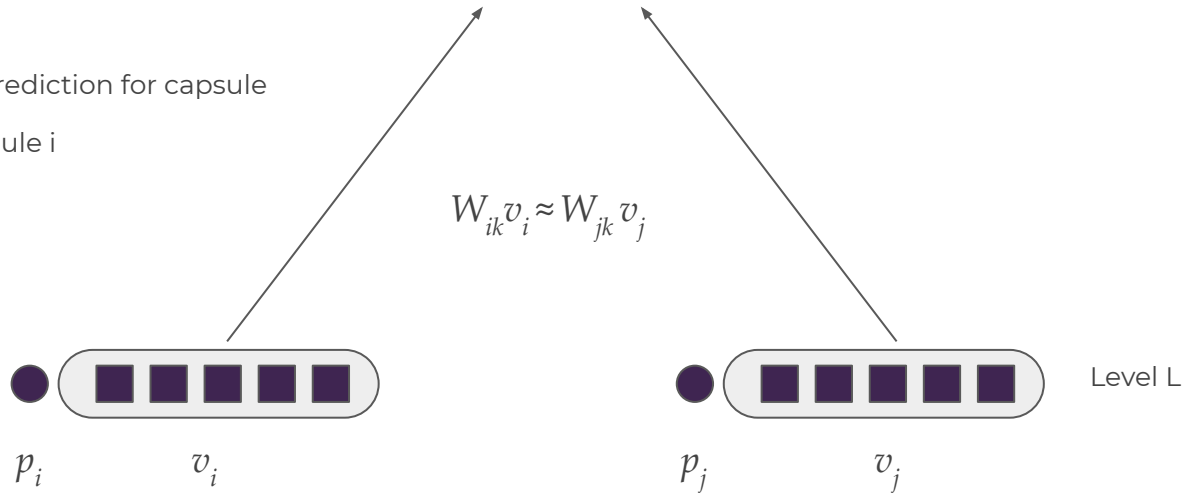


Fig 22. How to compute the pose of a high-level capsule.

Capsules

From raw pixels we obtain the poses of the parts, we multiply this poses by transformation matrices W (our weight matrices learnt through backpropagation) to get the predictions for the pose of the whole. We leverage these prediction to get the correct pose of the whole.

W_{ik} = viewpoint-invariant weight matrix

$\hat{u}_{k|i} = W_{ik} v_i$ = prediction for capsule pose k from capsule i

$v_k = c_{ik}\hat{u}_{k|i} + c_{jk}\hat{u}_{k|j}$

c_{ik} = coupling coefficient

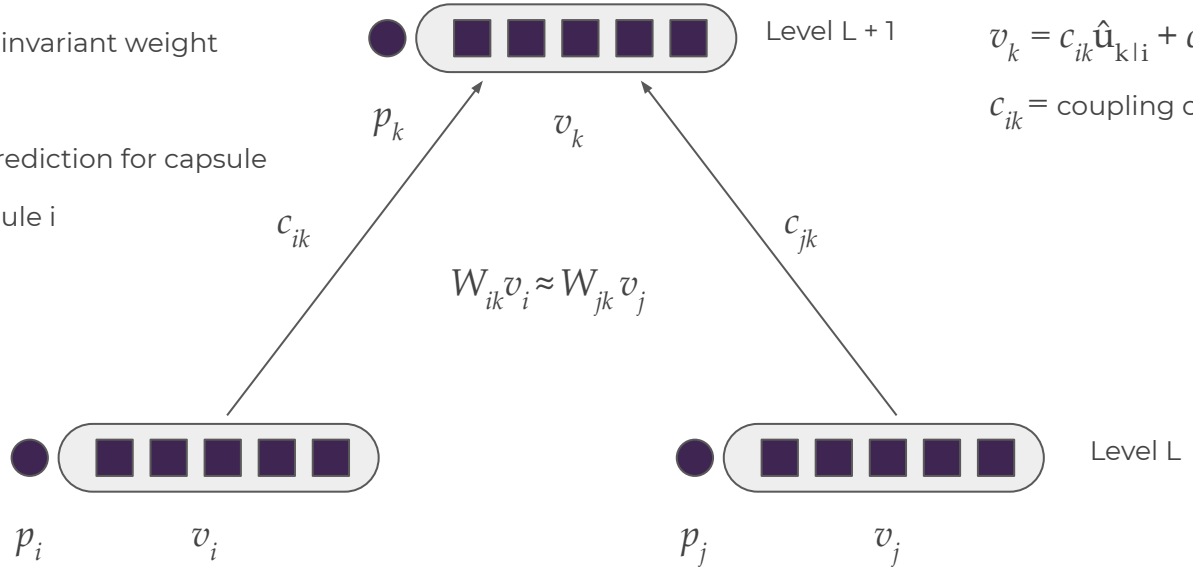


Fig 22. How to compute the pose of a high-level capsule.

Capsules

If a higher-order capsule finds a **cluster of similar predictions** from capsules in the layer below for its pose then it outputs:

1. A **high probability** that the higher-level entity exists.
2. The **center of gravity of the cluster**, which is the generalized pose of that entity.

We can find these tight clusters by means of the **routing-by-agreement** mechanism.
It is based on the idea that high-dimensional coincidences do not happen by chance: we must thrown away irrelevant predictions and keep only the closest ones (**coincidence filtering**).

This way of recognizing shapes generalizes naturally across changes in viewpoint because all the **poses change in the same way so the coincidence remains**.

Routing-by-agreement intuition

Let's see how to compute visually the pose for a high-level capsule k. It's an iterative process.

N *iterations. Votes* $\hat{u}_{k|*}$ *are already computed.*

def ***routing****(N,* $\hat{u}_{k|*}$*):*

For n *in* $0..N$*:*

compute the agreements between the votes $\hat{u}_{k|*}$ *and old* v_k

update mean v_k

return v_k

Routing-by-agreement intuition

Let's see how to compute visually the pose for a high-level capsule k. It's an iterative process.

● = a single prediction, the more black the dot is, the higher is c_{*k}.

Iteration 1: all the c_{*k} are equals

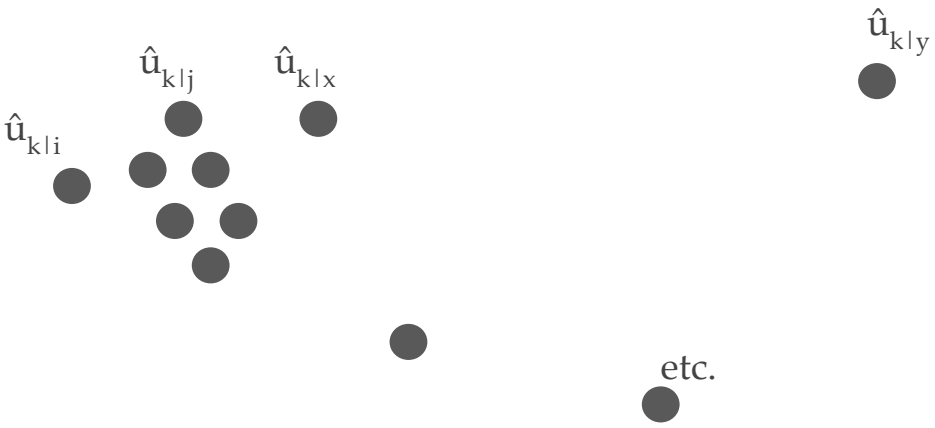


Fig 23. Routing by agreement.

Routing-by-agreement intuition

Let's see how to compute visually the pose for a high-level capsule k. It's an iterative process.

● = a single prediction, the more black the dot is, the higher is c_{*k}.

Iteration 1: compute mean

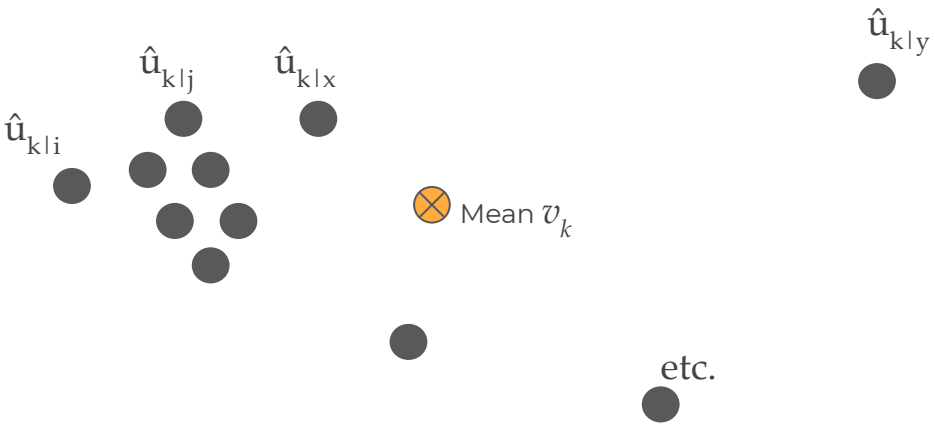


Fig 23. Routing by agreement.

Routing-by-agreement intuition

Let's see how to compute visually the pose for a high-level capsule k. It's an iterative process.

= a single prediction, the more black the dot is, the higher is c_{*k}.

Iteration 1: update all the c_{*k}

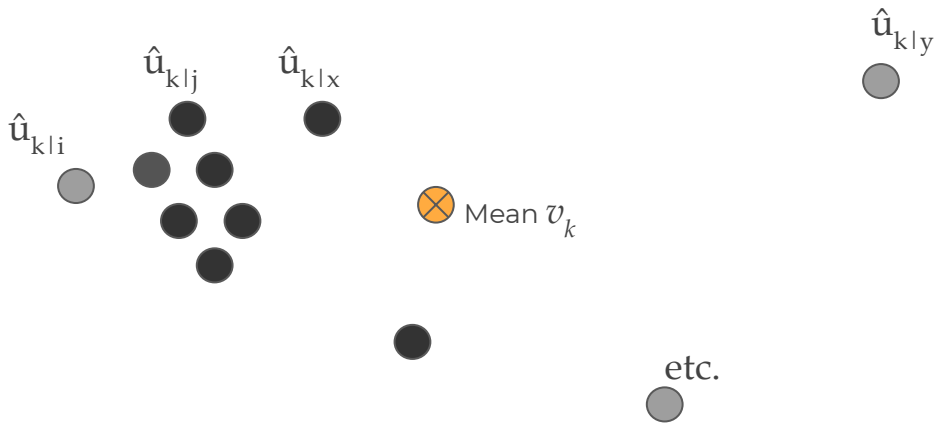


Fig 23. Routing by agreement.

Routing-by-agreement intuition

Let's see how to compute visually the pose for a high-level capsule k. It's an iterative process.

● = a single prediction, the more black the dot is, the higher is c_{*k}.

Iteration 2: compute mean

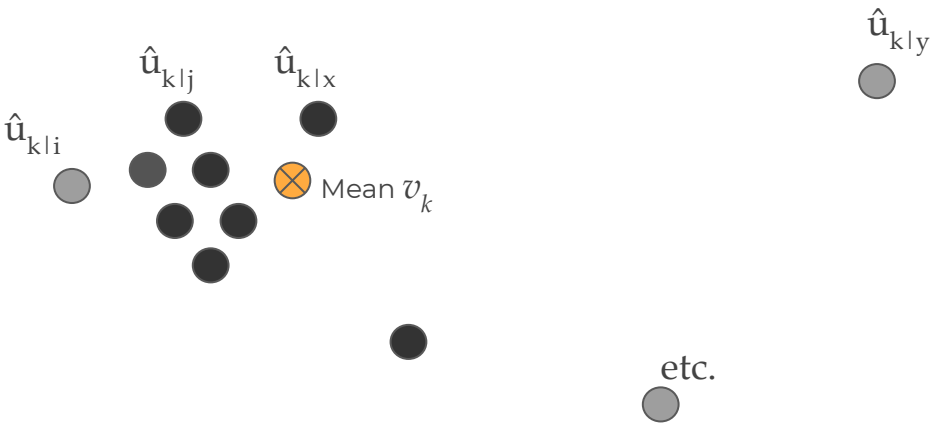


Fig 23. Routing by agreement.

Routing-by-agreement intuition

Let's see how to compute visually the pose for a high-level capsule k. It's an iterative process.

= a single prediction, the more black the dot is, the higher is c_{*k}.

Iteration 2: update all the c_{*k}

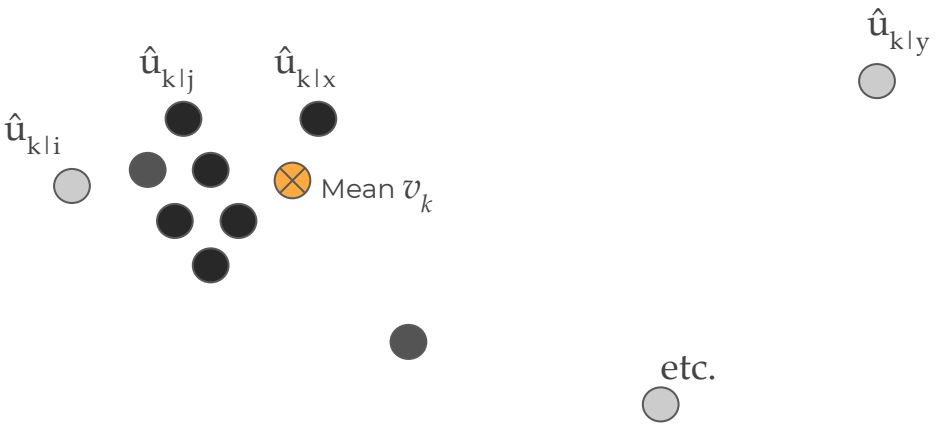


Fig 23. Routing by agreement.

Routing-by-agreement intuition

Let's see how to compute visually the pose for a high-level capsule k. It's an iterative process.

● = a single prediction, the more black the dot is, the higher is c_{*k}.

Iteration 3: compute mean

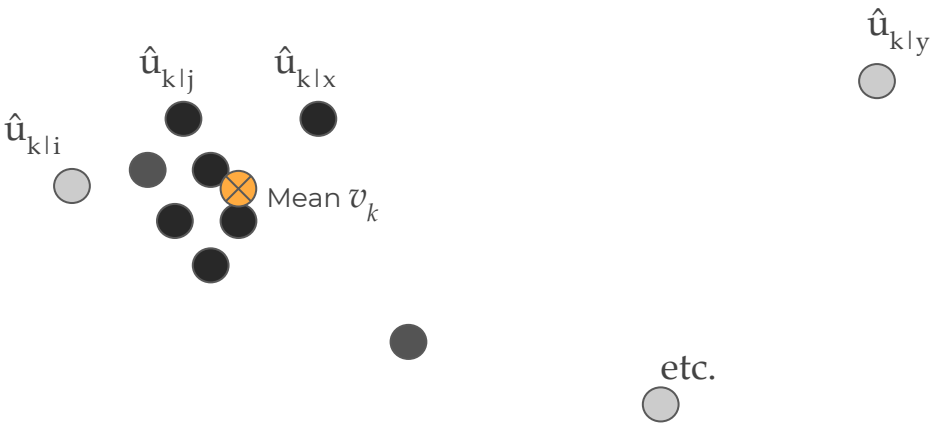


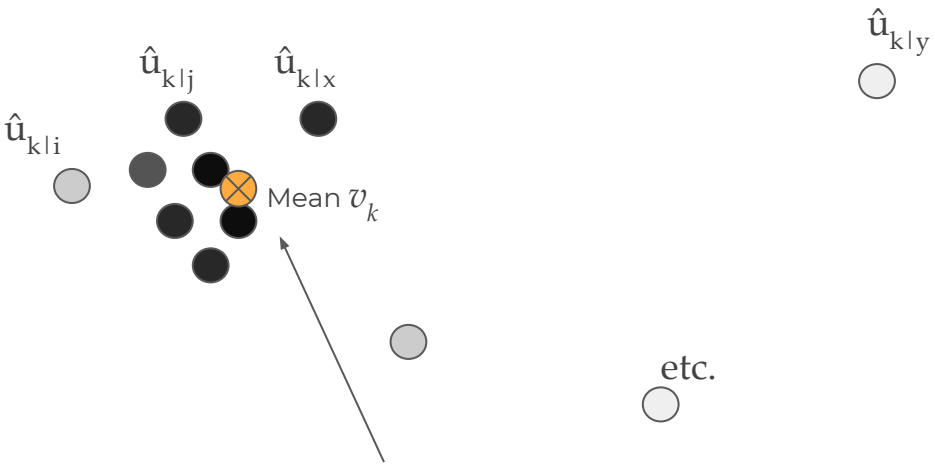
Fig 23. Routing by agreement.

Routing-by-agreement intuition

Let's see how to compute visually the pose for a high-level capsule k. It's an iterative process.

● = a single prediction, the more black the dot is, the higher is c_{*k}.

Iteration 3: compute mean



Final pose of high-level capsule k

Fig 23. Routing by agreement.

04

Vector Capsule Networks (VCapsNet)

VCapsNet

In [3], a capsule is implemented as an activity vector whose overall length represents the probability of existence of the entity and its orientation represents its instantiation parameters.

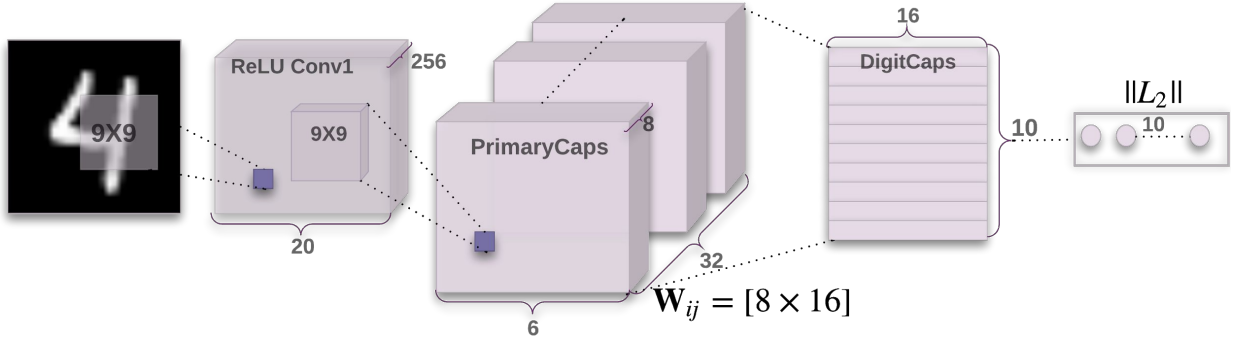


Fig 24. VCapsNet architecture (encoder part) [3].

Capsules are organized in types and layers. Capsules of the same type detects the same entity but in different positions.

Forward pass

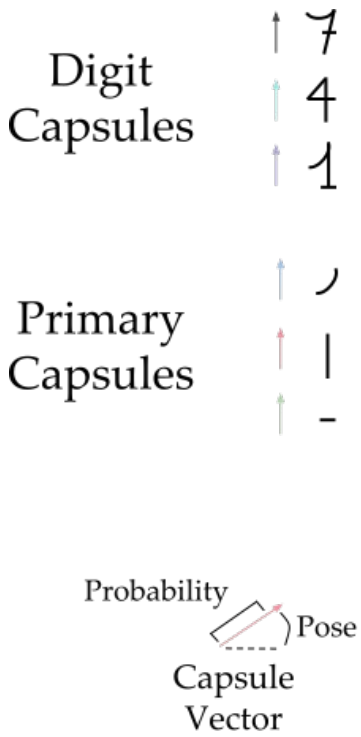


Fig 25. VCapsNet forward pass example.

Forward pass

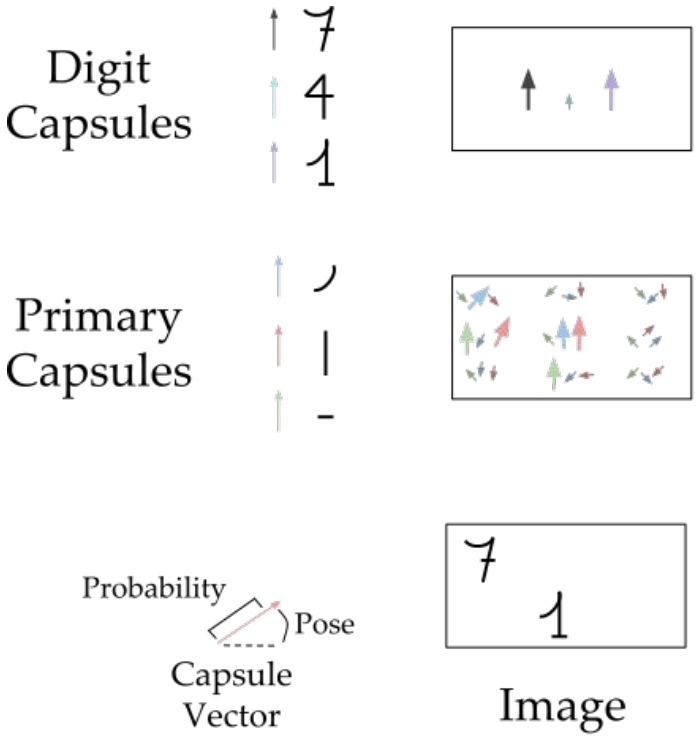


Fig 25. VCapsNet forward pass example.

Forward pass

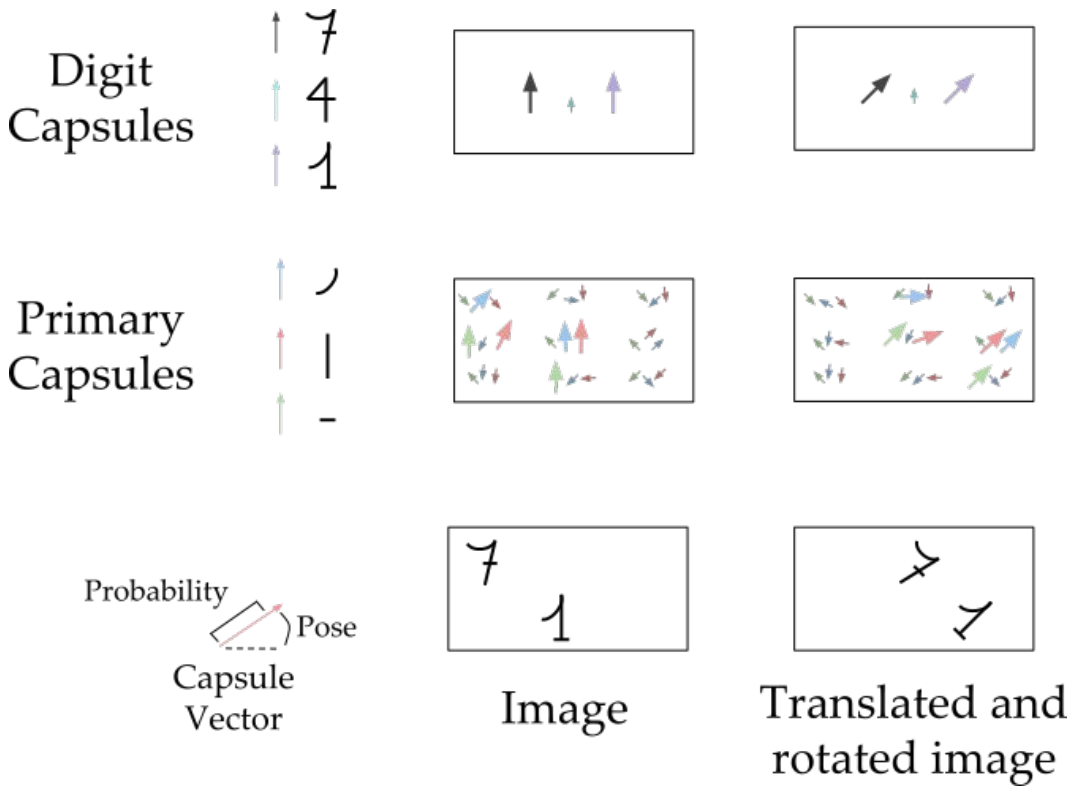


Fig 25. VCapsNet forward pass example.

Dynamic routing

$$\texttt{squash}(\mathbf{s}_j) = \frac{||\mathbf{s}_j||^2}{1+||\mathbf{s}_j||^2}\frac{\mathbf{s}_j}{||\mathbf{s}_j||} = \mathbf{v}_j$$

$$\texttt{softmax}(b_{ij}) = \frac{\exp(b_{ij})}{\sum_k \exp(b_{ik})} = c_{ij}$$

Procedure 1 Routing algorithm.

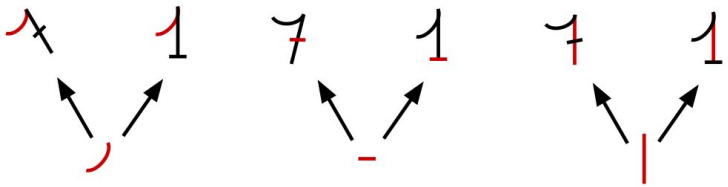
1: **procedure** ROUTING($\hat{\boldsymbol{u}}_{j|i}$, r, l)
2: for all capsule i in layer l and capsule j in layer $(l+1)$: $b_{ij} \leftarrow 0$.
3: **for** r iterations **do**
4: for all capsule i in layer l: $\mathbf{c}_i \leftarrow \texttt{softmax}(\mathbf{b}_i)$
5: for all capsule j in layer $(l+1)$: $\mathbf{s}_j \leftarrow \sum_i c_{ij}\hat{\mathbf{u}}_{j|i}$
6: for all capsule j in layer $(l+1)$: $\mathbf{v}_j \leftarrow \texttt{squash}(\mathbf{s}_j)$
7: for all capsule i in layer l and capsule j in layer $(l+1)$: $b_{ij} \leftarrow b_{ij} + \hat{\mathbf{u}}_{j|i}.\mathbf{v}_j$
return $\mathbf{v}_j$

Dynamic routing

$\longrightarrow = c_{ij}$

Predictions $\hat{\mathbf{u}}_{j|i}$ for level L + 1

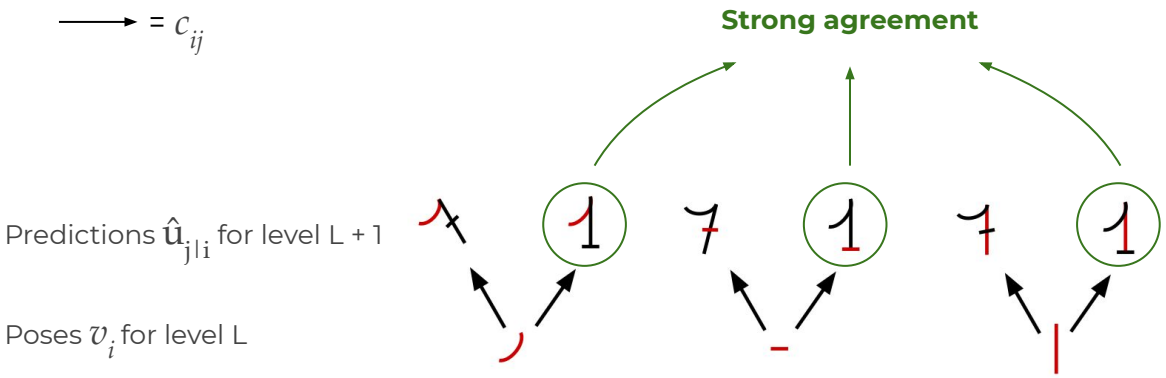
Poses v_i for level L



Procedure 1 Routing algorithm.

procedure ROUTING($\hat{\boldsymbol{u}}_{j|i}$, r, l)
  for all capsule i in layer l and capsule j in layer $(l+1)$: $b_{ij} \leftarrow 0$.
  for r iterations **do**
    for all capsule i in layer l: $\mathbf{c}_i \leftarrow \texttt{softmax}(\mathbf{b}_i)$
    for all capsule j in layer $(l+1)$: $\mathbf{s}_j \leftarrow \sum_i c_{ij}\hat{\mathbf{u}}_{j|i}$
    for all capsule j in layer $(l+1)$: $\mathbf{v}_j \leftarrow \texttt{squash}(\mathbf{s}_j)$
    for all capsule i in layer l and capsule j in layer $(l+1)$: $b_{ij} \leftarrow b_{ij} + \hat{\mathbf{u}}_{j|i}.\mathbf{v}_j$
  return $\mathbf{v}_j$

Dynamic routing



Procedure 1 Routing algorithm.

procedure ROUTING($\hat{\boldsymbol{u}}_{j|i}$, r, l)
  for all capsule i in layer l and capsule j in layer $(l+1)$: $b_{ij} \leftarrow 0$.
  for r iterations **do**
    for all capsule i in layer l: $\mathbf{c}_i \leftarrow \texttt{softmax}(\mathbf{b}_i)$
    for all capsule j in layer $(l+1)$: $\mathbf{s}_j \leftarrow \sum_i c_{ij}\hat{\mathbf{u}}_{j|i}$
    for all capsule j in layer $(l+1)$: $\mathbf{v}_j \leftarrow \texttt{squash}(\mathbf{s}_j)$
    for all capsule i in layer l and capsule j in layer $(l+1)$: $b_{ij} \leftarrow b_{ij} + \hat{\mathbf{u}}_{j|i}.\mathbf{v}_j$
  return $\mathbf{v}_j$

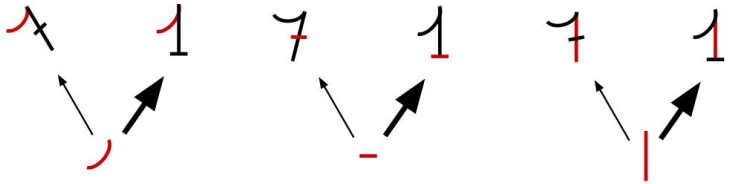
Dynamic routing

$\longrightarrow = c_{ij}$

We increase the coupling coefficients between low-level capsules and the capsule that represents "1".

Predictions $\hat{\mathbf{u}}_{j|i}$ for level L + 1

Poses v_i for level L

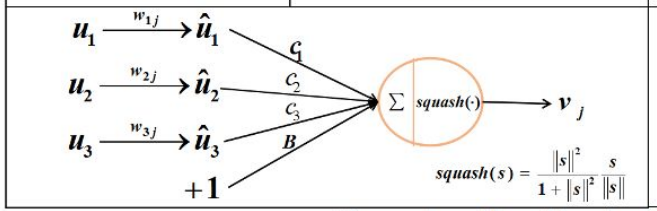


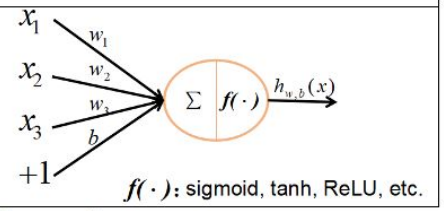
Procedure 1 Routing algorithm.

procedure ROUTING($\hat{\boldsymbol{u}}_{j|i}$, r, l)
  for all capsule i in layer l and capsule j in layer $(l+1)$: $b_{ij} \leftarrow 0$.
  for r iterations **do**
    for all capsule i in layer l: $\mathbf{c}_i \leftarrow \texttt{softmax}(\mathbf{b}_i)$
    for all capsule j in layer $(l+1)$: $\mathbf{s}_j \leftarrow \sum_i c_{ij}\hat{\mathbf{u}}_{j|i}$
    for all capsule j in layer $(l+1)$: $\mathbf{v}_j \leftarrow \texttt{squash}(\mathbf{s}_j)$
    for all capsule i in layer l and capsule j in layer $(l+1)$: $b_{ij} \leftarrow b_{ij} + \hat{\mathbf{u}}_{j|i}.\mathbf{v}_j$
  return $\mathbf{v}_j$

Capsule vs neuron

		capsule	VS.	traditional neuron
Input from low-level neurons/capsules		$\text{vector}(u_i)$		$\text{scalar}(x_i)$
Operations	Linear/Affine Transformation	$\hat{u}_{j\|i} = W_{ij} u_i + B_j$ (Eq. 2)		$a_{j\|i} = w_{ij} x_i + b_j$
	Weighting	$s_j = \sum_i c_{ij} \hat{u}_{j\|i}$ (Eq. 2)		$z_j = \sum_{i=1}^{3} 1 \cdot a_{j\|i}$
	Summation			
	Non-linearity activation	$v_j = squash(s_j)$ (Eq. 1)		$h_{w,b}(x) = f(z_j)$
output		$\text{vector}(v_j)$		$\text{scalar}(h)$





Capsule = New Version Neuron!
vector in, vector out VS. scalar in, scalar out

Fig 26. Capsule vs traditional neuron.

Full architecture

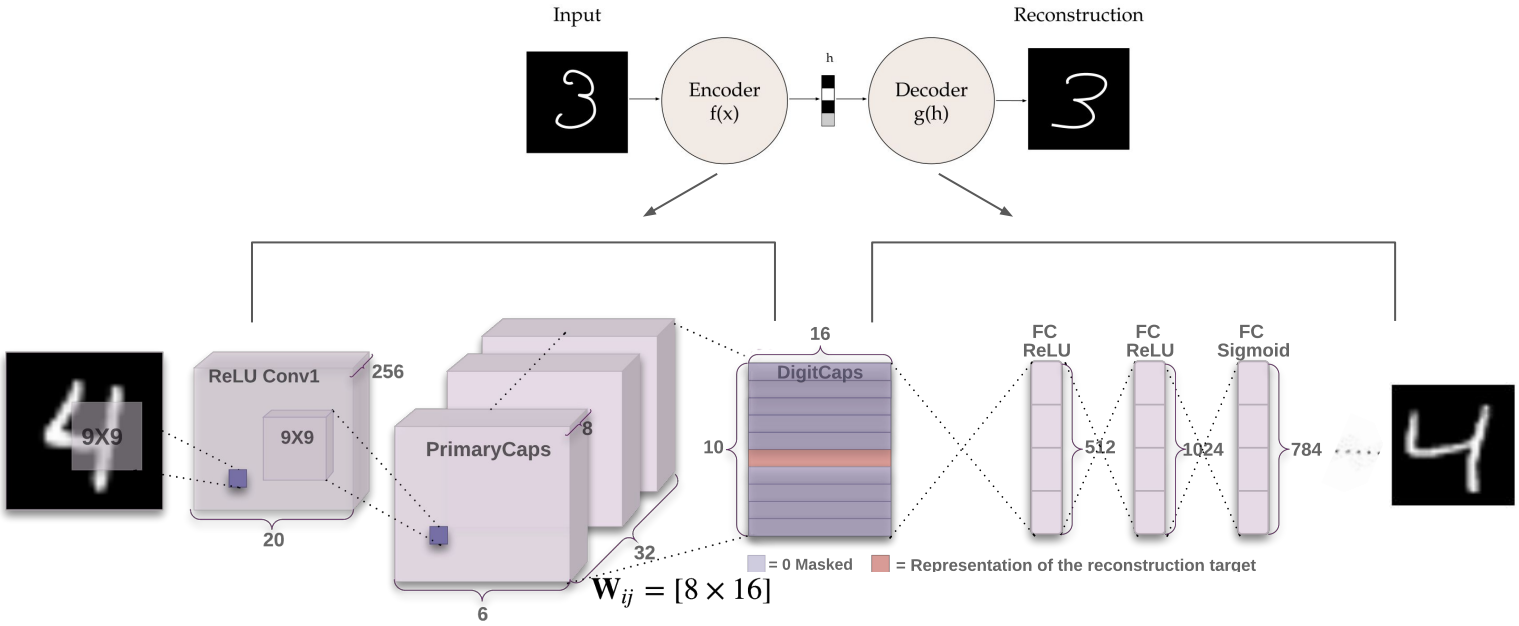


Fig 27. VCapsNet full architecture (encoder + decoder) [3].

The decoder encourages the digit capsules to encode the instantiation parameters of the input digit, it is a sort of regularization.

Loss

In order to train VCapsNets, two losses are used: the **margin loss** (encoder network) and the **reconstruction loss** (decoder network).

We have a separate margin loss L_k for each class k:

$$L_k = T_k \max(0, m^+ - \|\boldsymbol{v}_k\|)^2 + \lambda(1 - T_k)\max(0, \|\boldsymbol{v}_k\| - m^-)^2$$

where $T_k = 1$ if and only if digit k is present, $m^+ = 0.9$, $m^- = 0.1$ and $\lambda = 0.5$.

The reconstruction loss R is the sum of squared differences between the reconstructed image and the original one.

The total loss is defined as:

$$\text{Loss} = \sum_k L_k + 0.0005R$$

Backward pass

Standard backpropagation + Adam optimizer.

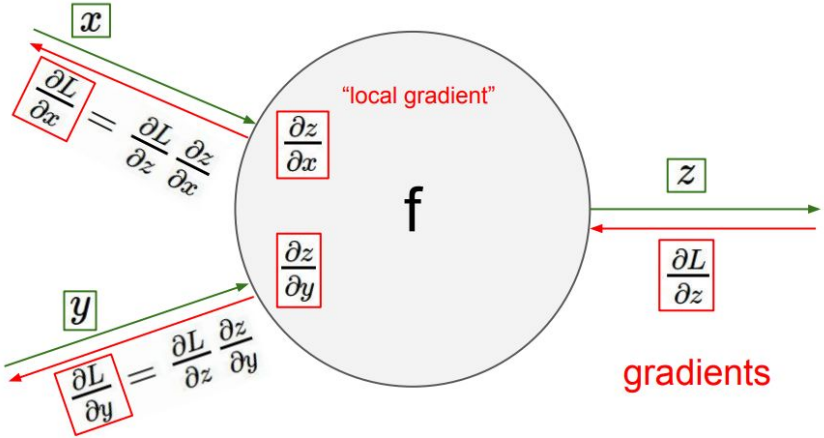


Fig 28. Backpropagation.
https://dongminlee.tistory.com/21

Dimension perturbations

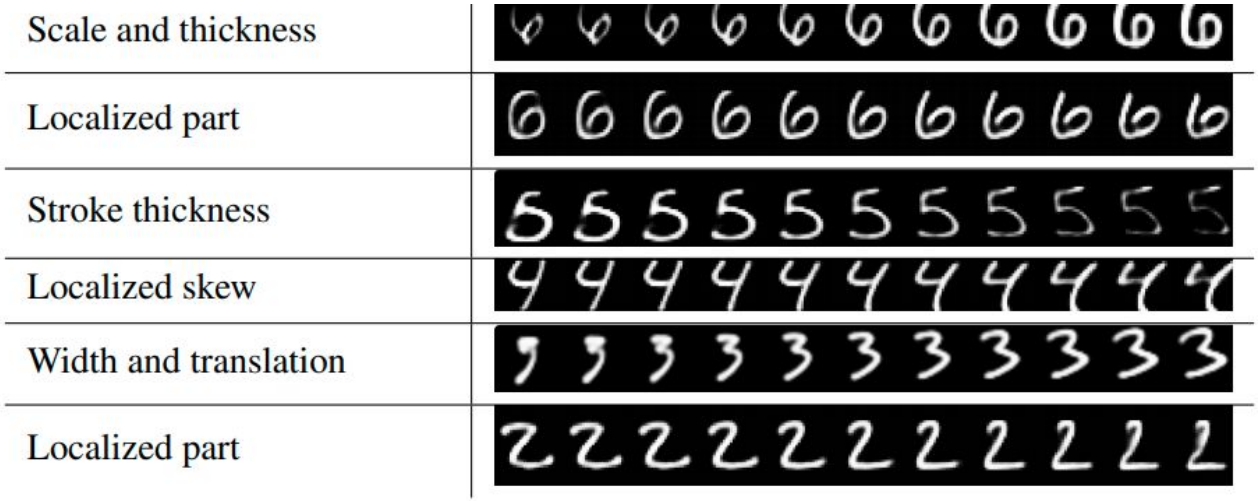


Fig 29. Dimension perturbations. Each row shows the reconstruction when one of the 16 dimensions in the DigitCaps representation is tweaked by intervals of 0.05 in the range [−0.25, 0.25].

Results

Method	Routing	Reconstruction	MNIST (%)	MultiMNIST (%)
Baseline	-	-	0.39	8.1
CapsNet	1	no	$0.34_{\pm 0.032}$	-
CapsNet	1	yes	$0.29_{\pm 0.011}$	7.5
CapsNet	3	no	$0.35_{\pm 0.036}$	-
CapsNet	3	yes	$\mathbf{0.25}_{\pm 0.005}$	**5.2**

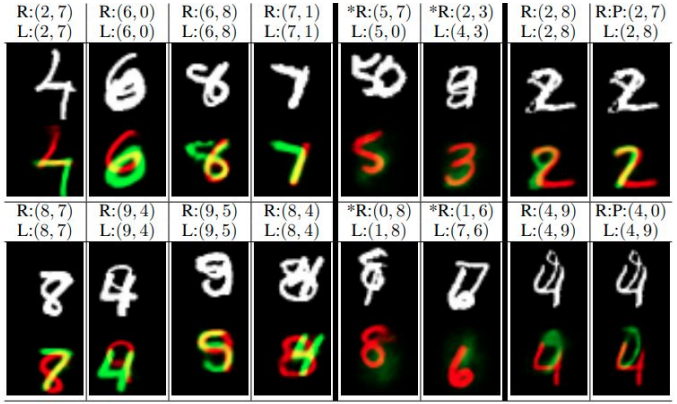
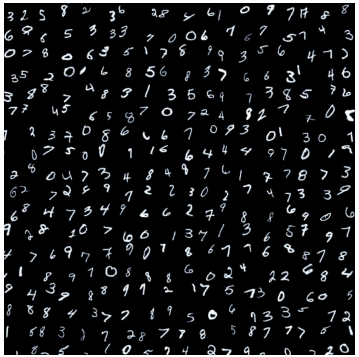
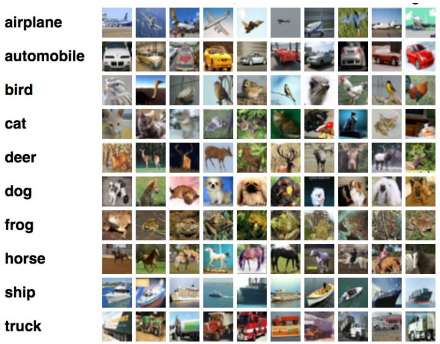


Fig 30. Baseline: 3 layers CNN, 256/256/128 chs + 2 layers FC: 35.4M params
VCapsNet: 8.2M params (6.8M encoder)

Results

Dataset	Test accuracy (%)
MNIST	99.75
affNIST	79.00
CIFAR10	89.40
smallNORB	97.30
SVHN	95.70





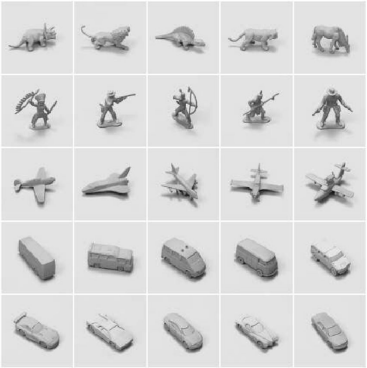


Fig 31. From left to right, MNIST, affNIST, CIFAR10, SVHN, smallNORB.
https://dongminlee.tistory.com/21

05

Matrix Capsule Networks (MCapsNet)

MCapsNet

Capsules are no longer vectors but rather 4x4 **matrices**. The advantage of this representation consists of having transformation matrices with n parameters instead of n^2 .

Each capsule has a **separated** logistic unit a to represent the probability of the presence of an entity.

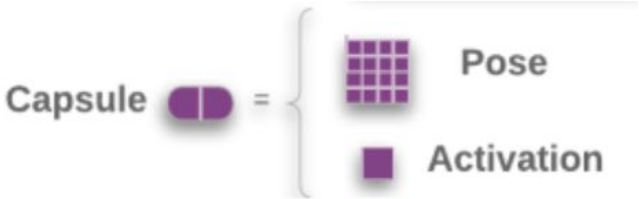


Fig 32. Matrix capsule [4].

EM Routing

The alternative method to do routing proposed in [4] is based on the expectation-maximization (EM) algorithm. The routing process has a strong resemblance to fitting a mixture of Gaussians using EM.

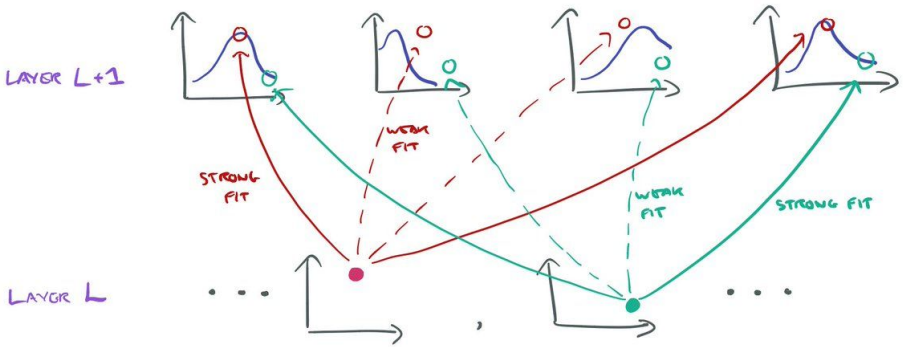


Fig 33. Fitting a mixture of Gaussians using EM.
https://blog.acolyer.org/2017/11/14/matrix-capsules-with-em-routing/

MCapsNet architecture

Convolutional capsule layers are introduced in order to build deeper capsule networks.

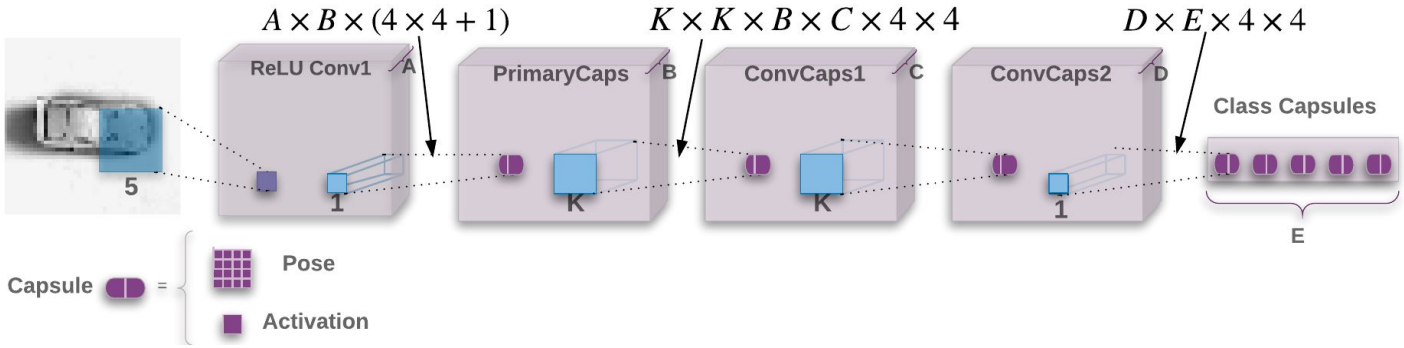


Fig 34. A matrix capsule network with 5 layers presented in [4]. The network has nearly 310K parameters.

06

Pros and cons

Pros

- Explicit notion of an **entity**.
- **Equivariance** in neural activities and **invariance** in weight matrices.
- Promising for image segmentation and object detection.
- **Routing** by agreement.
- **Robust** for affine transformations.
- Capsule are **easier to interpret** (rotation, thickness, skew, etc.).
- **Tree-structure**.

Cons

- Strong representational assumption: at each location in the image, there is at most one instance of the type of entity that a capsule represents. This phenomenon is known as **crowding** and it occurs also in human vision.
- Capsules, like other generative models, **account for everything** in the image so they have **poorer performances** when the **backgrounds** are too much **varied**, like in CIFAR10.
- **Training** capsule networks is **computationally expensive** because of the large transformation matrices and because of the **iterative routing algorithm which is executed during both training and testing time**.
- **Difficult to tune**. A capsule network has many hyperparameters like the number of types of capsules, their dimensions in every layer, the shape of the kernels etc.

References

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. **Gradient-based learning applied to document recognition**. In: *Proceedings of the IEEE 86.11 (1998),* pp. 2278–2324
- [2] G. E. Hinton, A. Krizhevsky, and S. D. Wang. **Transforming auto-encoders.** In: *Artificial Neural Networks and Machine Learning – ICANN 2011*, pages 44–51. Springer Berlin Heidelberg, 2011.
- [3] S. Sabour, N. Frosst, and G. E. Hinton. **Dynamic routing between capsules**. In: *Advances in Neural Information Processing Systems 30*, pages 3856–3866. 2017.
- [4] G. E. Hinton, S. Sabour, and N. Frosst (2018). **Matrix Capsules with EM routing**. In: *International Conference on Learning Representations*.

Additional resources

- Hinton: https://www.youtube.com/watch?v=x5Vxk9twXlE&t=1296s
- Aurélien Géron: https://www.youtube.com/watch?v=pPN8d0E3900&t=1s
- https://www.freecodecamp.org/news/understanding-capsule-networks-ais-alluring-new-architecture-bdb228173ddc/
- VCapsNet official code: https://github.com/Sarasra/models/tree/master/research/capsules
- MCapsNet official code: https://github.com/google-research/google-research/tree/master/capsule_em

THANKS

Does anyone have any questions?

Portrait of woman in d'hermine pass (Olga), 1923 by Pablo Picasso